LAPORAN PRAKTIKUM

INTERNET OF THINGS II

TUGAS 3



Nama :

Nama : Fadillah Akbar

NPM : 25782039

Kelas : TRI 3B

Dosen Pengampu : **Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.**

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, A.Md

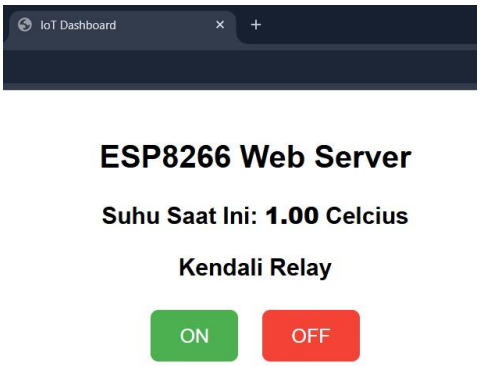
JURUSAN TEKNOLOGI INFORMASI

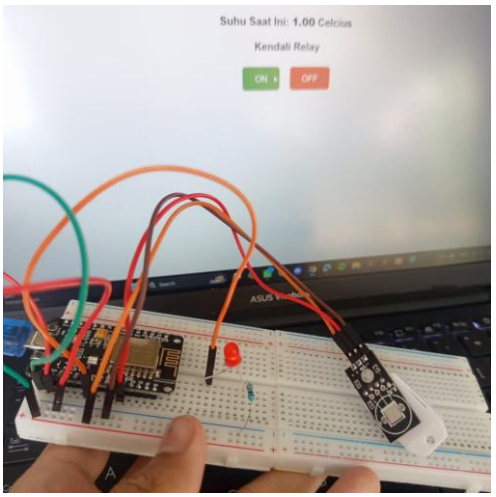
TEKNOLOGI REKAYASA INTERNET

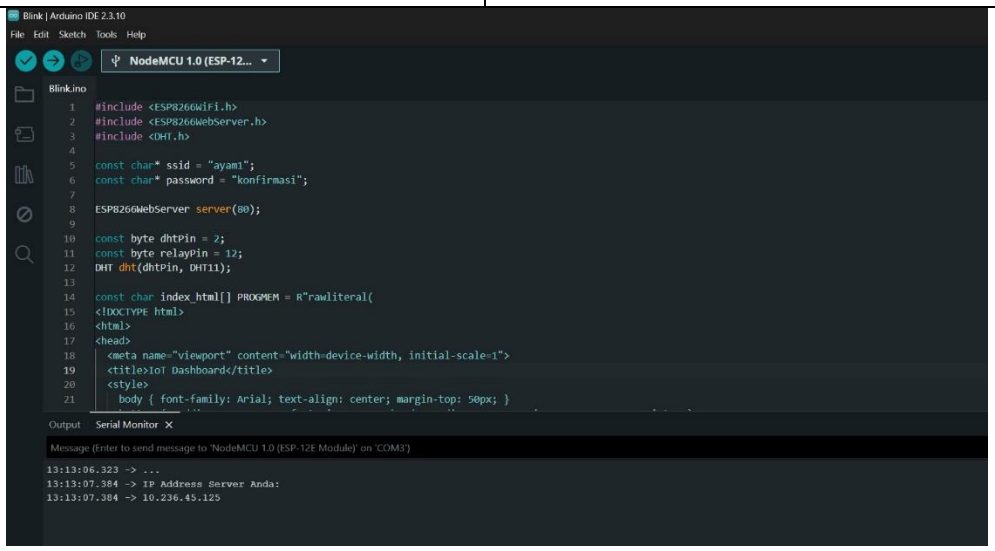
POLITEKNIK NEGERI LAMPUNG

TAHUN 202

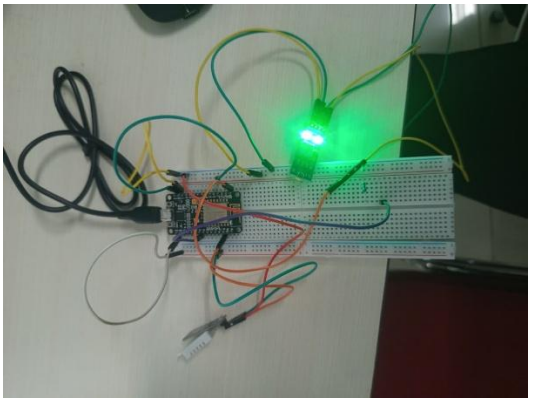
3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)



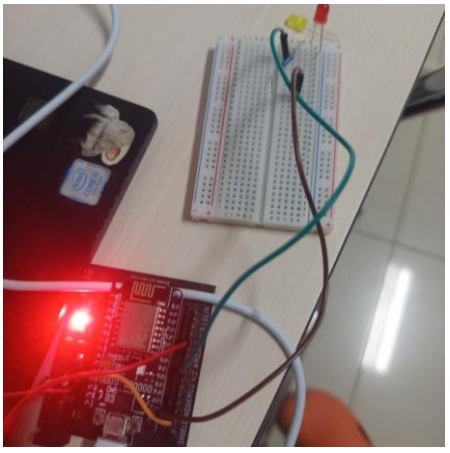




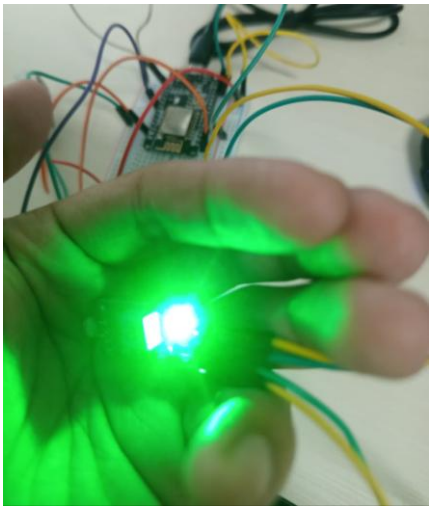
3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)



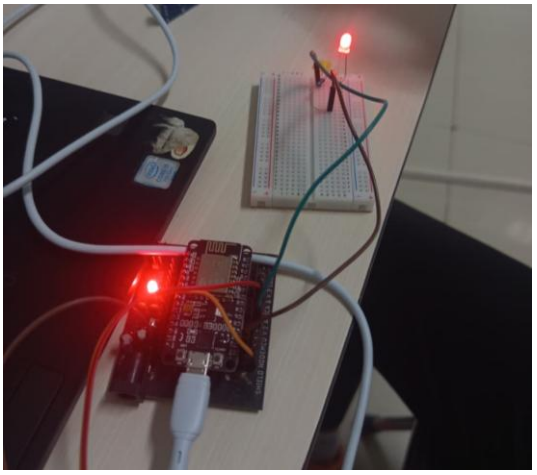
Client ketika blm ditutup LDR nya



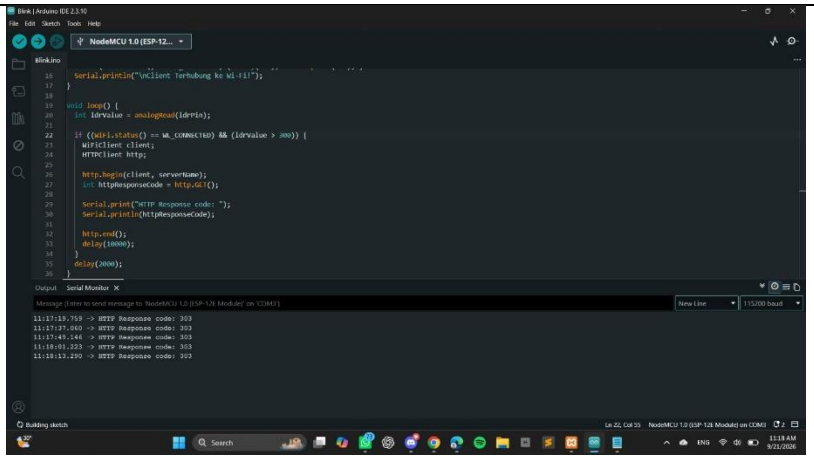
Server masih belum menyala karena LDR client belum di tutup



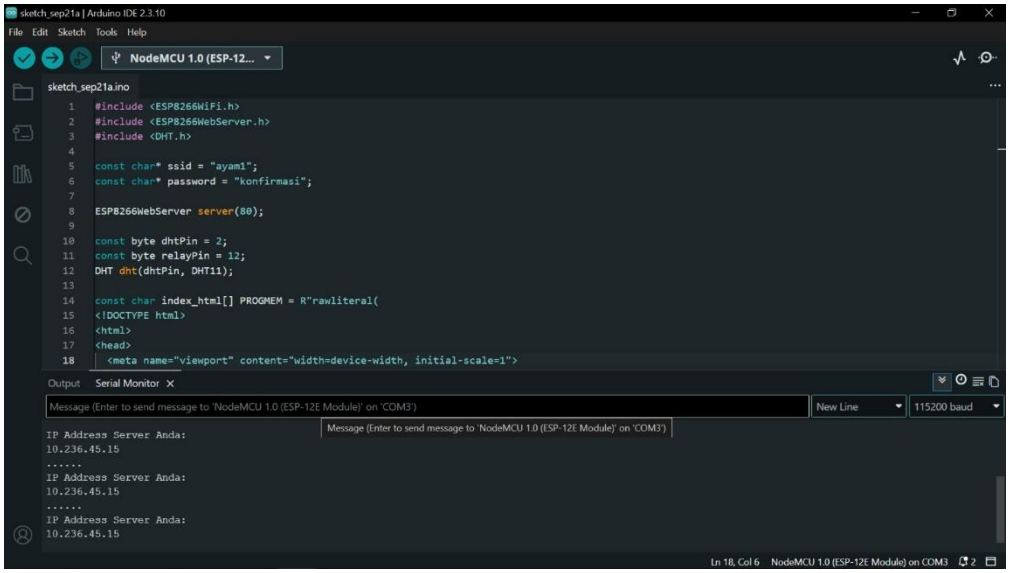
Menutup LDR client



Ketika LDR client di tutup, Server akan Menyala



Output Serial Monitor Client



Output Serial Monitor Server

3.6 Latihan

Kerjakan soal berikut secara mandiri untuk menguji penalaran dan modifikasi jaringan.

- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Jawab : Alasan menggunakan mode Station (STA) adalah agar kedua NodeMCU terhubung ke jaringan Wi-Fi atau hotspot yang sama sehingga berada dalam satu jaringan lokal (subnet) yang memungkinkan keduanya saling berkomunikasi dan mengirim data secara langsung.

- **Latihan 2 (Modifikasi Antarmuka & Telemetri):** Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:
    i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi dht.readHumidity(). Sisipkan placeholder %HUMIDITY% pada kerangka HTML di dalam tag (contoh: Kelembapan: %HUMIDITY% %), lalu perbarui fungsi handleRoot() menggunakan html.replace("%HUMIDITY%", ...) agar nilai kelembapan muncul di halaman *web*.
    ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi *Relay* saat ini:
        - Jika Relay sedang **OFF**, tombol menampilkan tulisan **"NYALAKAN (ON)"** (misal: warna hijau/biru) dan mengarahkan aksi ke /relay/on.
        - Jika Relay sedang **ON**, tombol otomatis berubah menjadi tulisan **"MATIKAN (OFF)"** (warna merah) dan mengarahkan aksi ke /relay/off.

        (Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti %RELAY_BUTTON%, lalu di fungsi handleRoot(),

periksa status pin relay menggunakan digitalRead(relayPin) sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).

Tuliskan potongan baris kode HTML dan fungsi handleRoot() yang Anda ubah!

Jawab :

1. **Modifikasi Variabel index_html**

   <meta http-equiv="refresh" content="5">

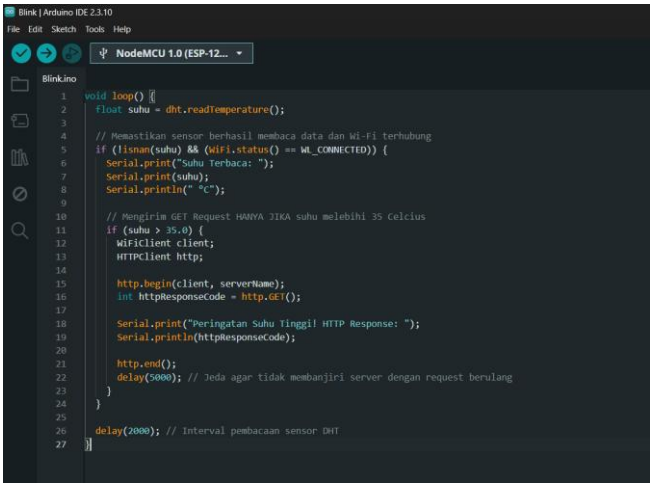
- **Latihan 3 (Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

Jawab : Jika NodeMCU Server mendadak kehilangan daya (*offline*), Serial Monitor pada Client akan mencetak angka nilai negatif:

HTTP Response code: -1

- **Latihan 4 (Pengembangan Algoritma):** Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

Jawab :



3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Antarmuka (*User Interface*) IoT tidak boleh statis. Saat ini, *Web Server* kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (*Web Dashboard*): Anda akan bertindak sebagai *Full-Stack IoT Developer* untuk memutakhirkan kapabilitas *Web Server* pada Latihan 2, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

- Tujuan: Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*).
- Tingkat Kesulitan: Menengah
- Instruksi:
    i. Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Latihan 2).
    ii. Modifikasi variabel Raw Literal index_html. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan *tag* <head> untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.
    iii. *Hint (Petunjuk):* Sintaks standar yang harus disisipkan adalah <meta http-equiv="refresh" content="5">.
    iv. Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.
- Instruksi Analisis Arsitektur:
    i. Setelah Auto-Refresh berjalan, cobalah tekan tombol *ON / OFF* (kendali LED) dari *browser*.
    ii. Apa kelemahan visual/interaksi (*flickering*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai *Real-Time Dashboard*? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth*, layar berkedip putih, atau potensi bentrokan jika Anda mengeklik tepat di detik ke-5).
- Kriteria Keberhasilan:
    - Nilai suhu dan kelembapan di *browser* otomatis berganti seiring kondisi

lingkungan tanpa perlu mengeklik tombol *refresh* di *browser*.

- Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas

